PES UNIVERSITY
Department of Computer Science and Engineering
UE23CS351A – DBMS Mini Project (Level 3) | AUG – DEC 2026

1. Title: Smart EV Charging Station Reservation System

Mini Project Report • SQL (MySQL) + NoSQL (MongoDB) • Next.js **2.**

2. Students

Student Name	SRN
Sriniket Deeduvanu	PES2UG24CS521
Surya R Naik	PES2UG24CS540

GitHub repository: https://github.com/sriniket-d/smart-ev-charging-reservation

3. Abstract

The Smart EV Charging Station Reservation System (branded “VoltReserve” in the UI) lets electric-vehicle owners register their vehicles, browse charging stations and reserve a charging point for a chosen time slot, while administrators manage stations, charging points and tariffs. The central business rule is that a charging point can never hold two overlapping active reservations. This is enforced in three layers: a server-side check under a row lock, database triggers, and CHECK/foreign-key constraints.

MySQL 8 (InnoDB) stores the eight relational entities (users, EVs, stations, charging points, tariffs, reservations, charging sessions, payments) and demonstrates keys, constraints, indexes, a view, triggers, joins, aggregation, subqueries and ACID transactions. A reservation and its pending payment are inserted in one transaction. MongoDB stores an append-only audit stream (activity_logs) of logins, reservation events and admin station changes, with indexes and an aggregation pipeline. Authentication uses bcrypt-hashed passwords and signed session cookies with customer and admin roles. Database settings come from environment variables. **4. Tech Stack**

Layer	Technology (as used in the repository)
Frontend	Next.js 16.4.2 (App Router), React 19.3.0, TypeScript 5, Tailwind CSS 4
Backend / API	Next.js Route Handlers (app/api/*)
Relational DB	MySQL 8 / InnoDB via mysql2 3.24 (connection pool, prepared statements)
NoSQL DB	MongoDB (local or Atlas) via the official mongodb Node.js driver 7.7
Authentication	bcryptjs (password hashing), jose (signed session token), HttpOnly cookie

Configuration	Environment variables (.env.local; template in .env.example)
Version control	Git + GitHub (both members have commits)

Database justification: MySQL holds structured, related data that needs keys, constraints, joins and transactions. MongoDB holds flexible, append-oriented activity events whose metadata varies by action. A vector database is not used because no semantic-search requirement exists in this domain.

5. SQL – Database Commands

All commands below are taken from db/*.sql and the Next.js Route Handlers (file names are given in each caption).

5.1 Database creation (db/schema.sql)

```
CREATE DATABASE IF NOT EXISTS smart_ev_charging_test
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_0900_ai_ci;

USE smart_ev_charging_test;
```

5.2 Table creation with keys and constraints (db/schema.sql)

Eight tables with PRIMARY KEY, FOREIGN KEY (ON UPDATE CASCADE / ON DELETE RESTRICT), UNIQUE, NOT NULL, DEFAULT, ENUM and CHECK constraints, plus indexes on reservations and payments.

```
CREATE TABLE users ( user_id INT AUTO_INCREMENT
    PRIMARY KEY, name VARCHAR(100) NOT NULL,
    email VARCHAR(255) NOT NULL UNIQUE, phone
    VARCHAR(20) NOT NULL UNIQUE, password_hash
    VARCHAR(255) NOT NULL, address VARCHAR(255),
    role ENUM('customer', 'admin') NOT NULL DEFAULT 'customer', created_at
    TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;
CREATE TABLE evs ( ev_id INT AUTO_INCREMENT PRIMARY
    KEY, user_id INT NOT NULL, vehicle_number
    VARCHAR(20) NOT NULL UNIQUE, model VARCHAR(100)
    NOT NULL, battery_capacity DECIMAL(6,2) NOT
    NULL, connector_type VARCHAR(50) NOT NULL,
    CONSTRAINT chk_evs_battery_capacity CHECK (battery_capacity > 0),
    CONSTRAINT fk_evs_user FOREIGN KEY (user_id)
        REFERENCES users(user_id)
        ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB;
CREATE TABLE charging_stations ( station_id INT
    AUTO_INCREMENT PRIMARY KEY, station_name
    VARCHAR(150) NOT NULL, location VARCHAR(255)
    NOT NULL, city VARCHAR(100) NOT NULL, latitude
    DECIMAL(10,7), longitude DECIMAL(10,7),
    CONSTRAINT chk_station_latitude
        CHECK (latitude IS NULL OR latitude BETWEEN -90 AND 90), CONSTRAINT
    chk_station_longitude
        CHECK (longitude IS NULL OR longitude BETWEEN -180 AND 180)
) ENGINE=InnoDB;
CREATE TABLE charging_points ( point_id INT
    AUTO_INCREMENT PRIMARY KEY, point_code
    VARCHAR(20) NOT NULL, station_id INT NOT
    NULL,
    connector_type VARCHAR(50) NOT NULL, power_kw
    DECIMAL(6,2) NOT NULL,
    status ENUM('Available', 'Occupied', 'Unavailable', 'Maintenance') NOT
        NULL DEFAULT 'Available',
    CONSTRAINT chk_point_power CHECK (power_kw > 0),
    CONSTRAINT uq_charging_points_point_code UNIQUE (point_code),
    CONSTRAINT fk_points_station FOREIGN KEY (station_id)
        REFERENCES charging_stations(station_id)
        ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB;
CREATE TABLE tariff_plans ( tariff_id INT
    AUTO_INCREMENT PRIMARY KEY, station_id INT NOT
    NULL, rate_per_kwh DECIMAL(8,2) NOT NULL,
    CONSTRAINT chk_tariff_rate CHECK (rate_per_kwh >= 0),
```

```
    CONSTRAINT fk_tariffs_station FOREIGN KEY (station_id)
        REFERENCES charging_stations(station_id)
        ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB;
CREATE  TABLE  reservations  (  reservation_id  INT
    AUTO_INCREMENT  PRIMARY  KEY,  user_id  INT  NOT
    NULL,  ev_id  INT  NOT  NULL,  point_id  INT  NOT
    NULL,  tariff_id  INT  NOT  NULL,  start_time
    DATETIME NOT NULL, end_time DATETIME NOT NULL,
    status ENUM('Booked', 'Confirmed', 'Cancelled', 'Completed', 'No-show')
        NOT NULL DEFAULT 'Booked', created_at TIMESTAMP NOT NULL
    DEFAULT  CURRENT_TIMESTAMP,  CONSTRAINT  chk_reservation_time
    CHECK (end_time > start_time),
    CONSTRAINT fk_reservations_user FOREIGN KEY (user_id)
        REFERENCES users(user_id)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_reservations_ev FOREIGN KEY (ev_id)
        REFERENCES evs(ev_id)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_reservations_point FOREIGN KEY (point_id)
        REFERENCES charging_points(point_id)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_reservations_tariff FOREIGN KEY (tariff_id)
        REFERENCES tariff_plans(tariff_id)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    INDEX idx_reservations_user_start (user_id, start_time),
    INDEX idx_reservations_point_time (point_id, start_time, end_time)
) ENGINE=InnoDB;
CREATE  TABLE  charging_sessions  (  session_id
    INT    AUTO_INCREMENT    PRIMARY    KEY,
    reservation_id   INT   NOT   NULL   UNIQUE,
    actual_start     DATETIME,     actual_end
    DATETIME,  energy_delivered  DECIMAL(8,2),
    total_cost    DECIMAL(10,2),    CONSTRAINT
    chk_session_energy
        CHECK (energy_delivered IS NULL OR energy_delivered >= 0),
    CONSTRAINT chk_session_cost
        CHECK (total_cost IS NULL OR total_cost >= 0),
    CONSTRAINT chk_session_time
        CHECK (actual_end IS NULL OR actual_start IS NULL
               OR actual_end >= actual_start),
    CONSTRAINT fk_sessions_reservation FOREIGN KEY (reservation_id)
        REFERENCES reservations(reservation_id)
        ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB;
CREATE   TABLE   payments   (   payment_id   INT
    AUTO_INCREMENT PRIMARY KEY, reservation_id
    INT  NOT  NULL,  amount  DECIMAL(10,2)  NOT
    NULL, payment_method VARCHAR(50) NOT NULL,
    payment_status ENUM('Pending', 'Completed', 'Failed', 'Refunded')
        NOT NULL DEFAULT 'Pending', payment_time TIMESTAMP NOT
    NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT chk_payment_amount CHECK (amount >= 0),
    CONSTRAINT fk_payments_reservation FOREIGN KEY (reservation_id)
        REFERENCES reservations(reservation_id)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    INDEX idx_payments_reservation (reservation_id)
) ENGINE=InnoDB;
```

5.3 ALTER and DROP (db/migrations/…point_code.sql, db/schema.sql, db/advanced_queries.sql)

```
UPDATE charging_points
SET point_code = CONCAT('CP-', LPAD(point_id, 2, '0'))
WHERE point_code IS NULL OR TRIM(point_code) = '';
```

```
ALTER TABLE charging_points
    MODIFY point_code VARCHAR(20) NOT NULL;

DROP TEMPORARY TABLE IF EXISTS dbms_ddl_demo;

CREATE TEMPORARY TABLE dbms_ddl_demo (
    demo_id INT PRIMARY KEY,
    demo_value VARCHAR(50) NOT NULL
) ENGINE=InnoDB;

ALTER TABLE dbms_ddl_demo
    ADD COLUMN note VARCHAR(100) NULL;


DROP TEMPORARY TABLE dbms_ddl_demo;

DROP TABLE IF EXISTS payments;
DROP TABLE IF EXISTS charging_sessions;
-- ... (remaining tables dropped in reverse dependency order)
```

The migration also runs ALTER TABLE charging_points ADD COLUMN point_code … and CREATE UNIQUE INDEX uq_charging_points_point_code, each guarded by an information_schema check.

5.4 INSERT (db/seed.sql, app/api/reservations/route.ts)

```
INSERT INTO users
    (name, email, phone, password_hash, address, role)
VALUES
    ('Arjun Kumar', 'arjun.kumar@example.com', '9876543210',
     '$2b$12$REPLACE_WITH_REAL_BCRYPT_HASH', 'Electronic City, Bengaluru', 'customer'),
    ('Admin Demo', 'admin@example.com', '9876543211',
     '$2b$12$REPLACE_WITH_REAL_BCRYPT_HASH', 'Bengaluru', 'admin');

INSERT INTO reservations
    (user_id, ev_id, point_id, tariff_id, start_time, end_time, status)
VALUES
    (1, 1, 1, 1, '2026-11-05 10:00:00', '2026-11-05 11:00:00', 'Booked'),
    (1, 2, 2, 1, '2026-11-06 14:00:00', '2026-11-06 15:00:00', 'Confirmed');

      `INSERT INTO reservations
        (user_id, ev_id, point_id, tariff_id, start_time, end_time, status)
       VALUES (?, ?, ?, ?, ?, ?, 'Booked')`,

      `INSERT INTO payments
        (reservation_id, amount, payment_method, payment_status)
       VALUES (?, 0.00, 'Demo / pending', 'Pending')`,
```

5.5 SELECT with JOIN and ORDER BY (db/advanced_queries.sql)

```
SELECT
    r.reservation_id,
    u.name AS customer_name,
    e.vehicle_number,
    e.model AS vehicle_model,
    s.station_name,
    s.city,
    p.point_id,
    r.start_time,
    r.end_time,
    r.status
FROM reservations AS r
JOIN users AS u
    ON u.user_id = r.user_id
JOIN evs AS e
    ON e.ev_id = r.ev_id
JOIN charging_points AS p
    ON p.point_id = r.point_id
JOIN charging_stations AS s
    ON s.station_id = p.station_id
ORDER BY r.start time DESC;
```

5.6 GROUP BY, HAVING and aggregate functions (db/advanced_queries.sql)

```
SELECT
    s.station_id,
    s.station_name,
    COUNT(r.reservation_id) AS reservation_count,
    COALESCE(AVG(TIMESTAMPDIFF(MINUTE, r.start_time, r.end_time)), 0)
        AS average_duration_minutes
FROM charging_stations AS s
JOIN charging_points AS p
    ON p.station_id = s.station_id
LEFT JOIN reservations AS r
    ON r.point_id = p.point_id
GROUP BY s.station_id, s.station_name
HAVING COUNT(r.reservation_id) >= 1
ORDER BY reservation_count DESC, s.station_name;
```

5.7 Subquery (db/advanced_queries.sql)

```
SELECT
    s.station_id,
    s.station_name,
    s.city
FROM charging_stations AS s
WHERE s.station_id IN (
    SELECT p.station_id
    FROM charging_points AS p
    JOIN reservations AS r
        ON r.point_id = p.point_id
    WHERE r.status IN ('Booked', 'Confirmed')
)
ORDER BY s.city, s.station_name;
```

5.8 UPDATE and DELETE (application routes and DML demo)

```
      `UPDATE  reservations  SET
       status = 'Cancelled'
       WHERE reservation_id = ? AND user_id = ?`,
      [reservation_id, user.userId]


      `UPDATE evs
       SET vehicle_number = ?, model = ?, battery_capacity = ?, connector_type = ?
       WHERE ev_id = ? AND user_id = ?`,


"DELETE FROM evs WHERE ev_id = ? AND user_id = ?",


          `DELETE FROM charging_points
           WHERE point_id IN (${placeholders}) AND station_id = ?`,
```

db/advanced_queries.sql (lines 32–63) additionally runs INSERT, UPDATE, SELECT and DELETE on a temporary station inside START TRANSACTION … ROLLBACK, so no demo data is kept.

5.9 View (db/views.sql)

```
CREATE OR REPLACE VIEW advanced_reservation_details AS
SELECT
    r.reservation_id,
    r.user_id,
    u.name,
    r.ev_id,
    e.vehicle_number,
    r.point_id,
    p.station_id,
    s.station_name,
    r.tariff_id,
    r.start_time,
    r.end_time,
    r.status
FROM reservations AS r
JOIN users AS u
    ON u.user_id = r.user_id
JOIN evs AS e
```

5.10 Trigger – overlap prevention (db/triggers.sql)

```
DELIMITER //

DROP TRIGGER IF EXISTS reservations_prevent_overlap_insert//

CREATE TRIGGER reservations_prevent_overlap_insert
BEFORE INSERT ON reservations
FOR EACH ROW
BEGIN
    DECLARE conflict_count INT DEFAULT 0;

    IF NEW.status IN ('Booked', 'Confirmed') THEN
        SELECT COUNT(*)
        INTO conflict_count
        FROM reservations
        WHERE point_id = NEW.point_id
          AND status IN ('Booked', 'Confirmed')
          AND start_time < NEW.end_time
          AND end_time > NEW.start_time;

        IF conflict_count > 0 THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Overlapping reservation not allowed';
        END IF;
    END IF;
END//

DELIMITER ;
```

A matching BEFORE UPDATE trigger (reservations_prevent_overlap_update) is defined in the same file; it additionally excludes the row being updated (reservation_id <> OLD.reservation_id).

5.11 Transaction with row locking (app/api/reservations/route.ts)

```
await connection.beginTransaction();

const [points] = await connection.query(
  `SELECT station_id, status, connector_type
   FROM charging_points
   WHERE point_id = ?
   FOR UPDATE`,

const [conflicts] = await connection.query(
  `SELECT reservation_id
   FROM reservations
   WHERE point_id = ?
     AND status IN ('Booked', 'Confirmed')
     AND start_time < ?
     AND end_time > ?
   LIMIT 1
   FOR UPDATE`,
  [point_id, end, start]

  `INSERT INTO reservations
    (user_id, ev_id, point_id, tariff_id, start_time, end_time, status)
   VALUES (?, ?, ?, ?, ?, ?, 'Booked')`,

  `INSERT INTO payments
    (reservation_id, amount, payment_method, payment_status)
   VALUES (?, 0.00, 'Demo / pending', 'Pending')`,

await connection.commit();
```

If any check fails, connection.rollback() is called, so the reservation and its Pending payment are committed or discarded together. Admin station create/update/delete also use transactions.

5.12 Index inspection (db/advanced_queries.sql)

```
EXPLAIN
SHOW INDEX FROM reservations;
```

```
SELECT reservation_id, start_time, end_time, status
FROM reservations
WHERE user_id = 1
  AND start_time >= '2026-11-01 00:00:00'
ORDER BY start_time;

EXPLAIN
SELECT reservation_id, start_time, end_time, status
FROM reservations
WHERE point_id = 1
  AND start_time < '2026-11-06 12:00:00'
  AND end_time > '2026-11-06 10:00:00';
```

Not implemented in SQL: stored procedures/functions and FULLTEXT search (none exist in the repository).

6. NoSQL / Vector Database Commands

MongoDB (NoSQL): Implemented. Vector database: Not Implemented (not required for this project).

Database smart_ev (env MONGODB_DB), collection activity_logs. Document shape: action, userId, entityType, entityId, optional metadata, timestamp (docs/mongodb-data-model.md).

Index creation and insert (lib/mongodb.ts)

```
await collection.createIndex(
  { timestamp: -1 },
  { name: "idx_activity_logs_timestamp" }
);
await
collection.createIndex(
  { userId: 1, timestamp: -1 },
  { name: "idx_activity_logs_user_timestamp" }
);
await
collection.createIndex(
  { action: 1, timestamp: -1 },
  { name: "idx_activity_logs_action_timestamp" }
);
await
collection.insertOne({
  ...input, timestamp:
  new Date(),
});
```

Retrieval – customer sees own events, admin sees all (app/api/activity-logs/route.ts)

```
const filter = user.role === "admin" ? {} : { userId: user.userId };
const logs = await
db
  .collection("activity_logs")
  .find(filter)
  .sort({ timestamp: -1 })
  .limit(50)
  .toArray();
```

Aggregation pipeline (db/mongodb_activity_aggregation.js)

```
db.activity_logs.aggregate([
  {
    $group: {
      _id:          "$action",
      eventCount: { $sum: 1 }
    }
  },
  {
    $project: { _id:
      0,      action:
      "$_id",
      eventCount: 1
    }
  },
  {
    $sort: {
```

```
      action: 1
```

Logged actions: auth.login, reservation.created, reservation.cancelled, station.created, station.updated, station.deleted. Logging runs after the MySQL commit and is best-effort, so a MongoDB outage cannot undo a booking.

7. Next.js – Project Tree Structure

```
smart-ev-charging-reservation/
├── app/
│   ├── api/
│   │   ├── activity-logs/route.ts       (MongoDB activity retrieval)
│   │   ├── admin/stations/route.ts      (admin station/point/tariff management)
│   │   ├── auth/
│   │   │   ├── login/route.ts
│   │   │   ├── logout/route.ts
│   │   │   ├── register/route.ts
│   │   │   └── session/route.ts
│   │   ├── booking-options/route.ts
│   │   ├── db-test/route.ts
│   │   ├── evs/route.ts                 (My EV: list/add/edit/delete)
│   │   ├── reservations/route.ts        (create/list/cancel + transaction)
│   │   └── stations/route.ts
│   ├── login/page.tsx
│   ├── reservations/page.tsx
│   ├── layout.tsx
│   ├── page.tsx                         (main application UI)
│   ├── globals.css
│   └── favicon.ico
├── lib/
│   ├── auth.ts           (session sign/verify)
│   ├── server-auth.ts    (session + role checks)
│   ├── db.ts             (MySQL pool from env vars)
│   └── mongodb.ts        (MongoDB client + logActivity)
├── db/
│   ├── schema.sql
│   ├── seed.sql
│   ├── views.sql
│   ├── triggers.sql
│   ├── advanced_queries.sql
│   ├── mongodb_activity_aggregation.js
│   └── migrations/20261106_add_charging_point_code.sql
├── docs/  (architecture, ER diagram, data dictionary, MongoDB model,
│           project overview, test cases, demo checklist)
├── public/
├── .env.example
├── package.json
├── next.config.ts
├── tsconfig.json
├── eslint.config.mjs
├── TASK_DISTRIBUTION.md
└── README.md
```

Excluded: node_modules, .next, build output, .git and .env.local.

8. Sample Screenshots

Genuine screenshots could not be captured while preparing this report because the application, MySQL and MongoDB were not run in the report-preparation environment. No screenshot has been fabricated.

Figure 1 – Login / Create Account Page:

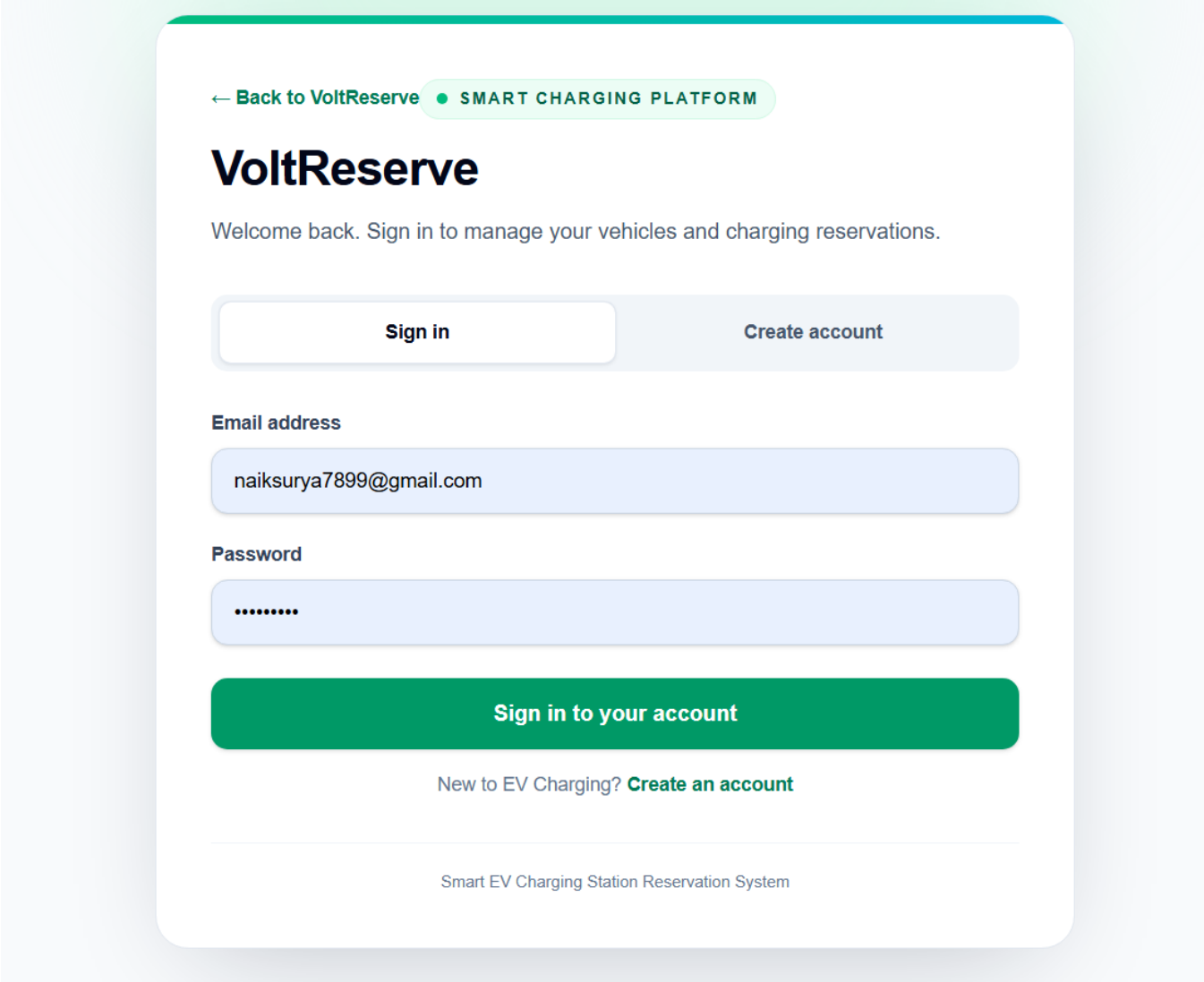

← Back to VoltReserve
SMART CHARGING PLATFORM
VoltReserve
Welcome back. Sign in to manage your vehicles and charging reservations.
Sign in
Create account
Email address
naiksurya7899@gmail.com
Password
••••••••••
Sign in to your account
New to EV Charging? Create an account
Smart EV Charging Station Reservation System

Figure 2 – Customer dashboard and my EV

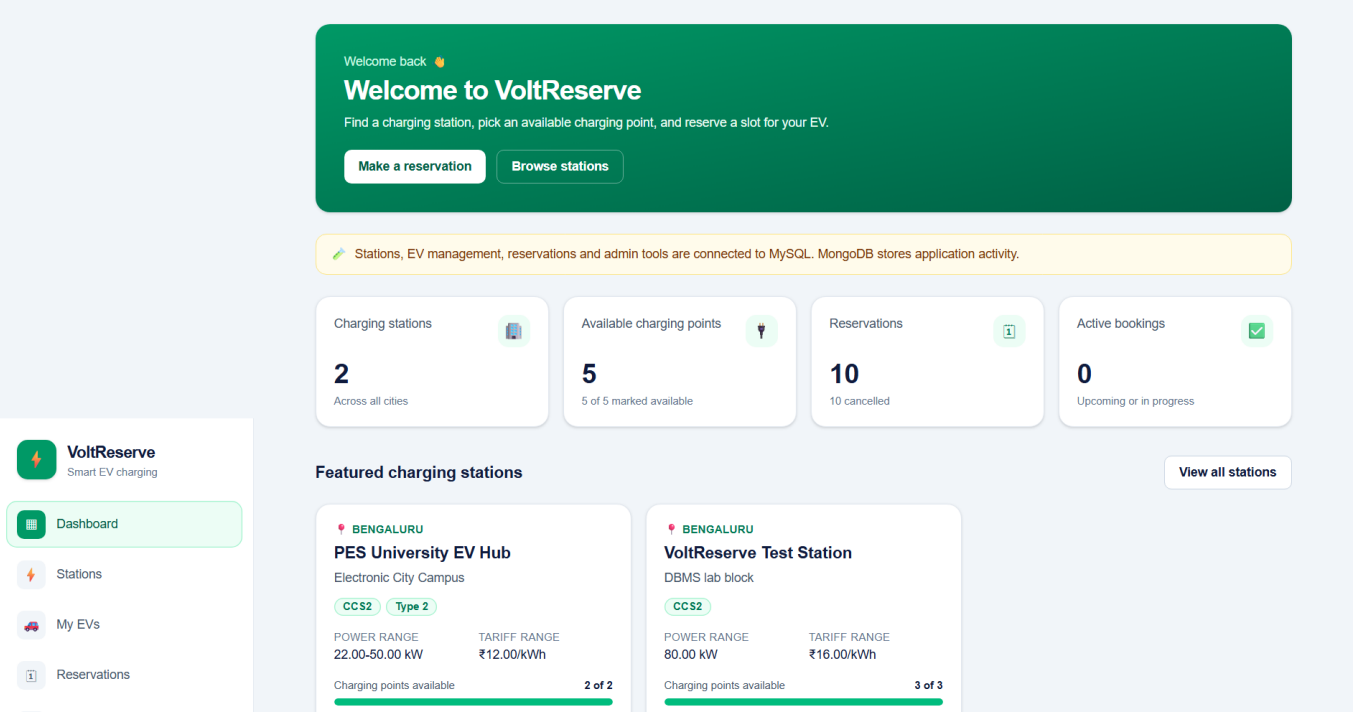
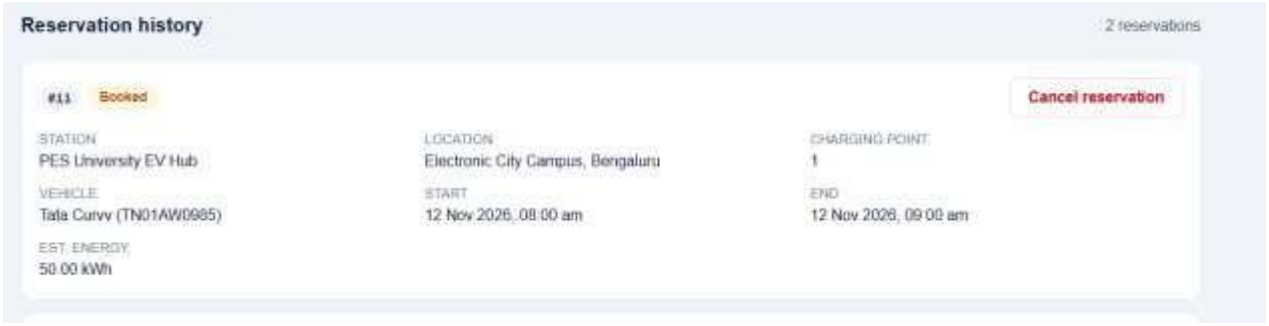


Figure 3 – Reservation Booking and Overlap rejection



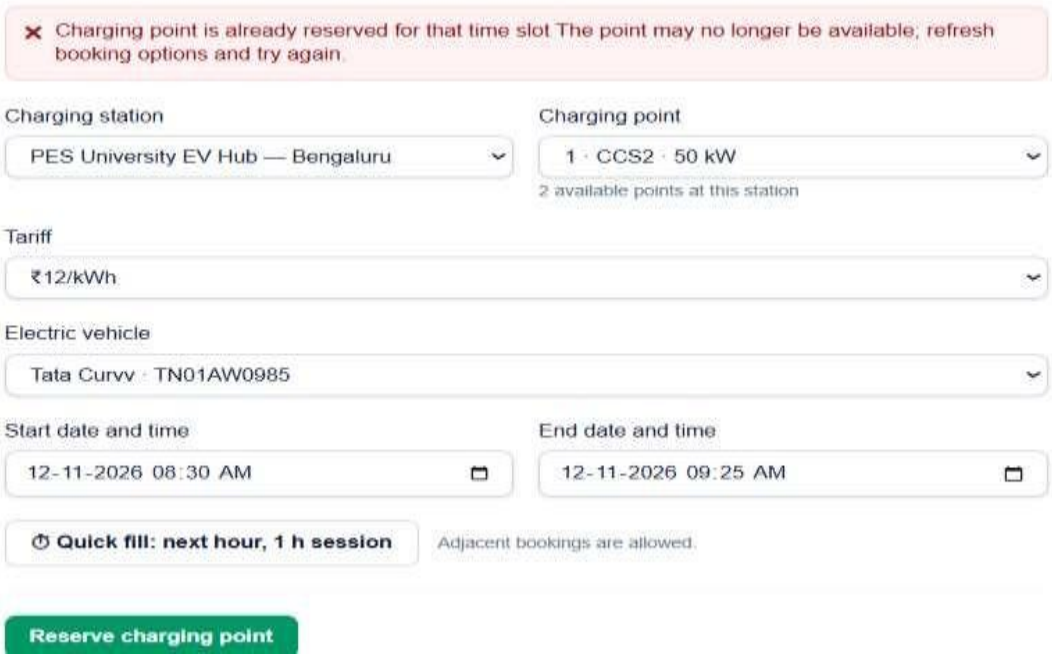


Figure 4 – Admin Station Management



Figure 5 – Activity Screen

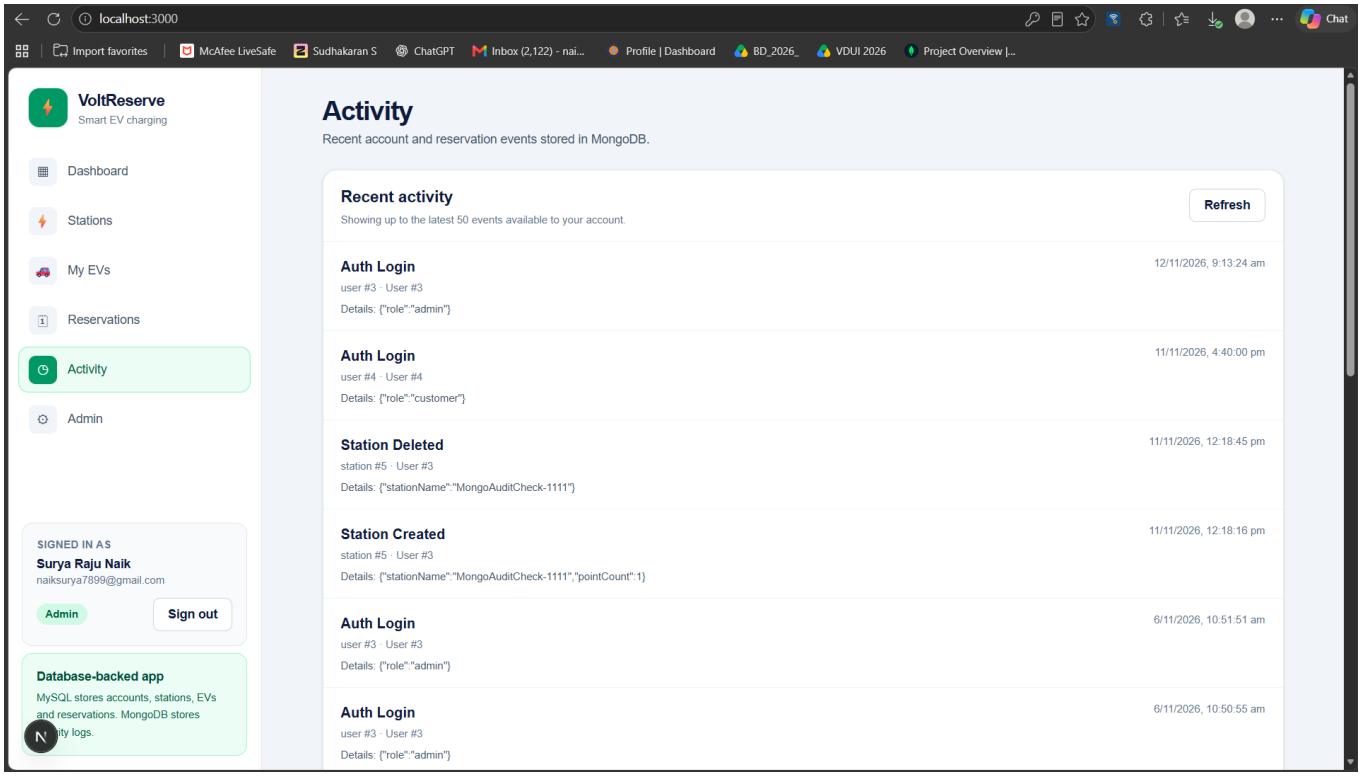


Figure 6 – MySQL Workbench: Tables and ER Diagram

```
mysql> USE smart_ev_charging_test;
Database changed
mysql> SHOW TABLES;
+-----------------------------------+
| Tables_in_smart_ev_charging_test  |
+-----------------------------------+
| advanced_reservation_details      |
| charging_points                   |
| charging_sessions                 |
| charging_stations                 |
| evs                               |
| payments                          |
| reservations                      |
| tariff_plans                      |
| users                             |
+-----------------------------------+
9 rows in set (0.01 sec)
```

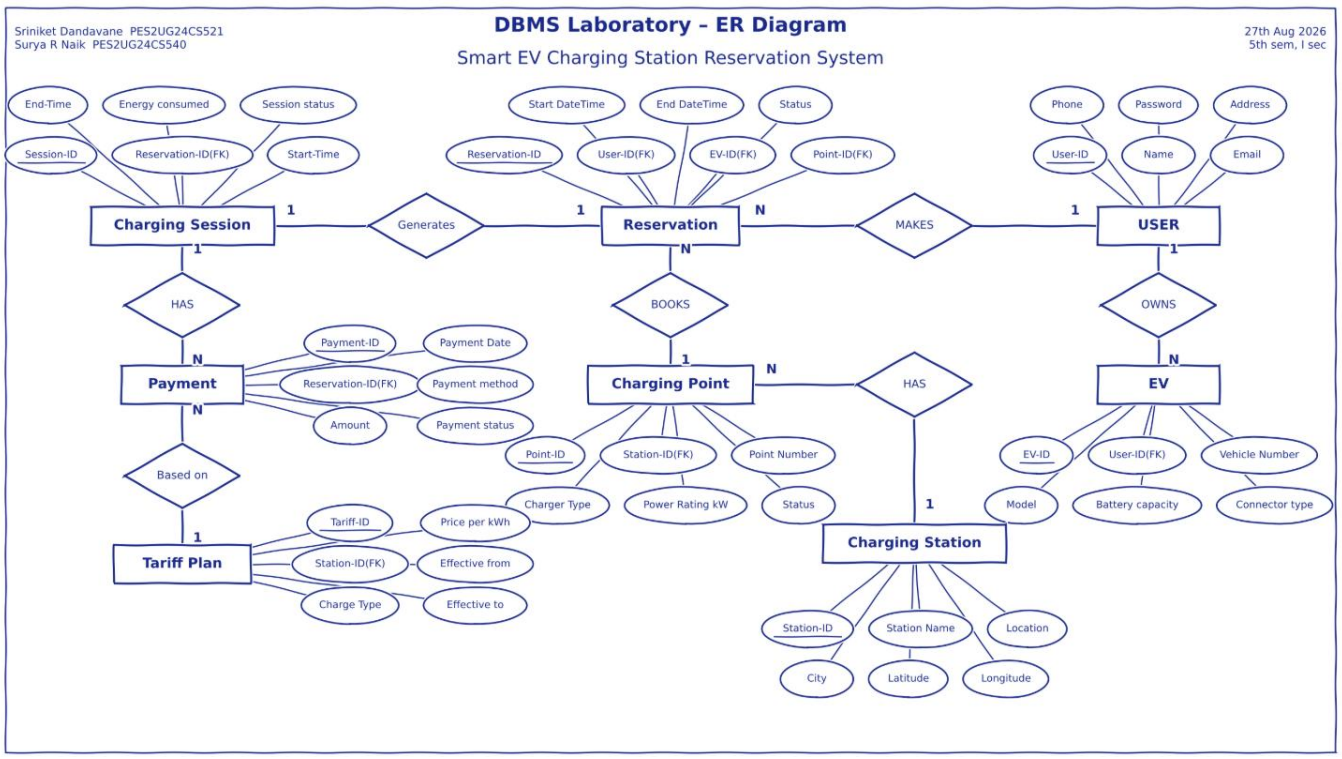


Figure 7 – MySQL query output

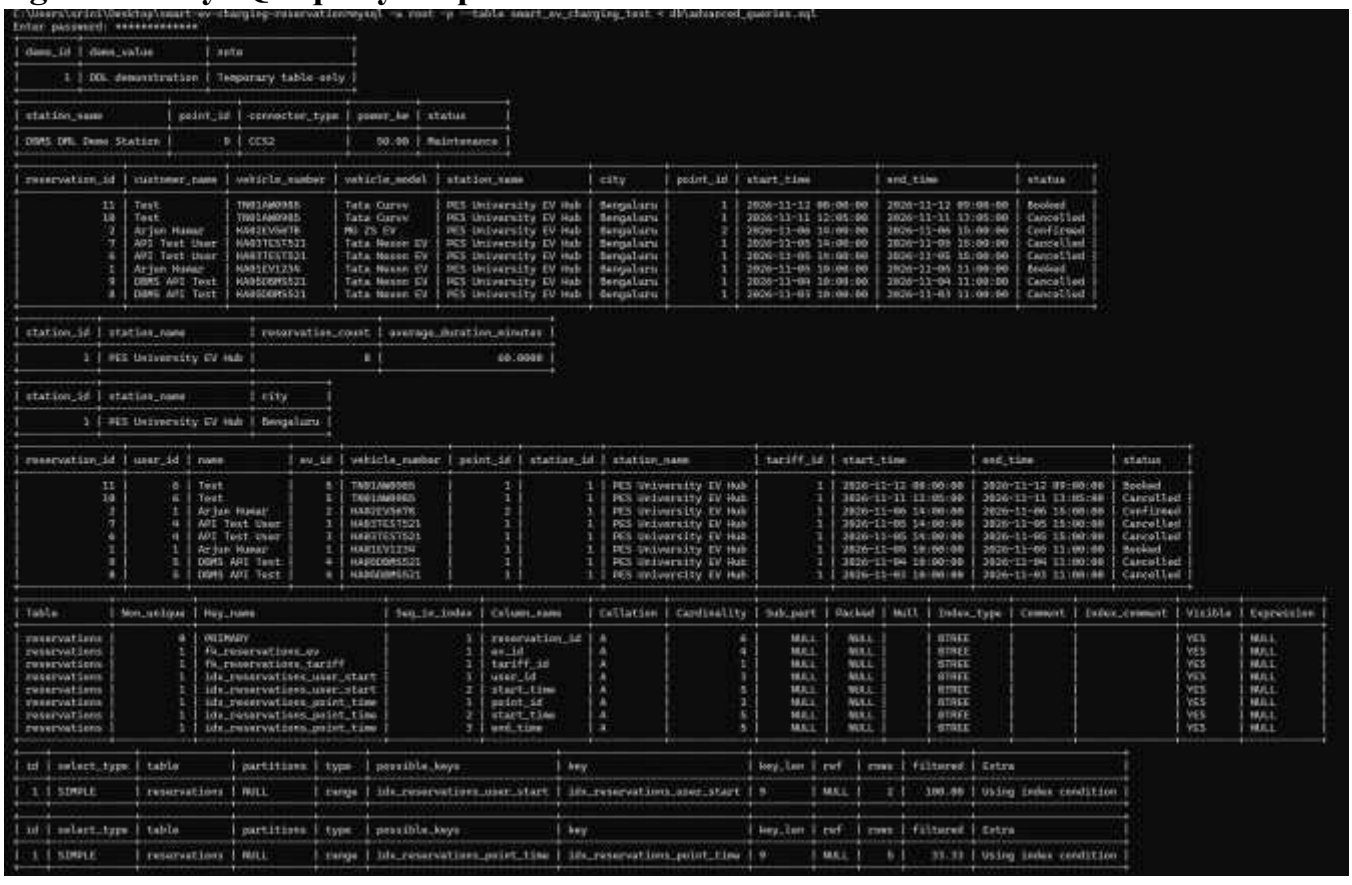


Figure 8 – Trigger Test

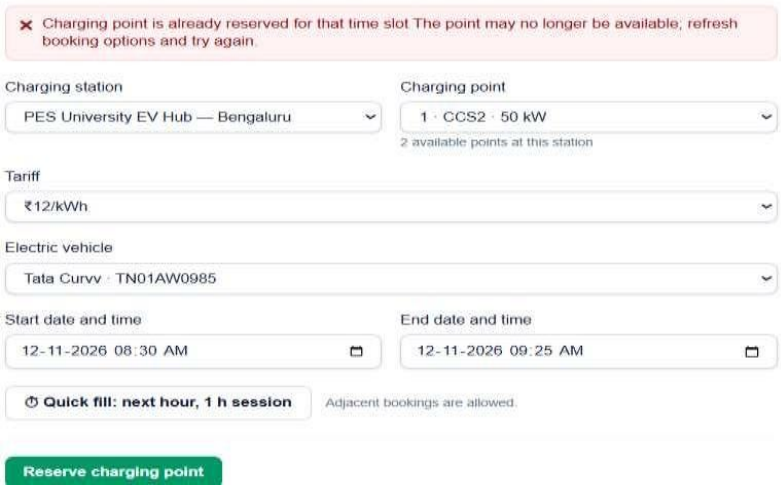


Figure 9 – MongoDB Compass

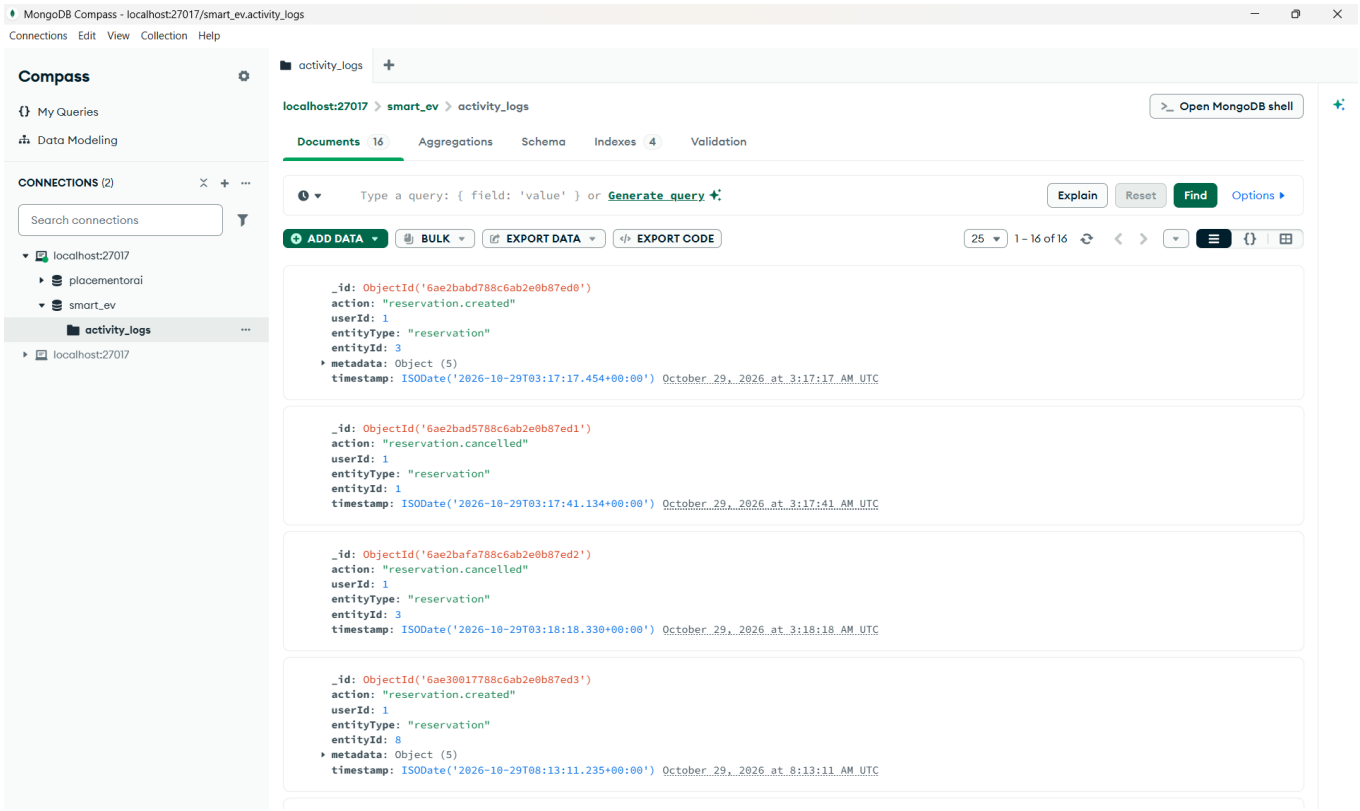
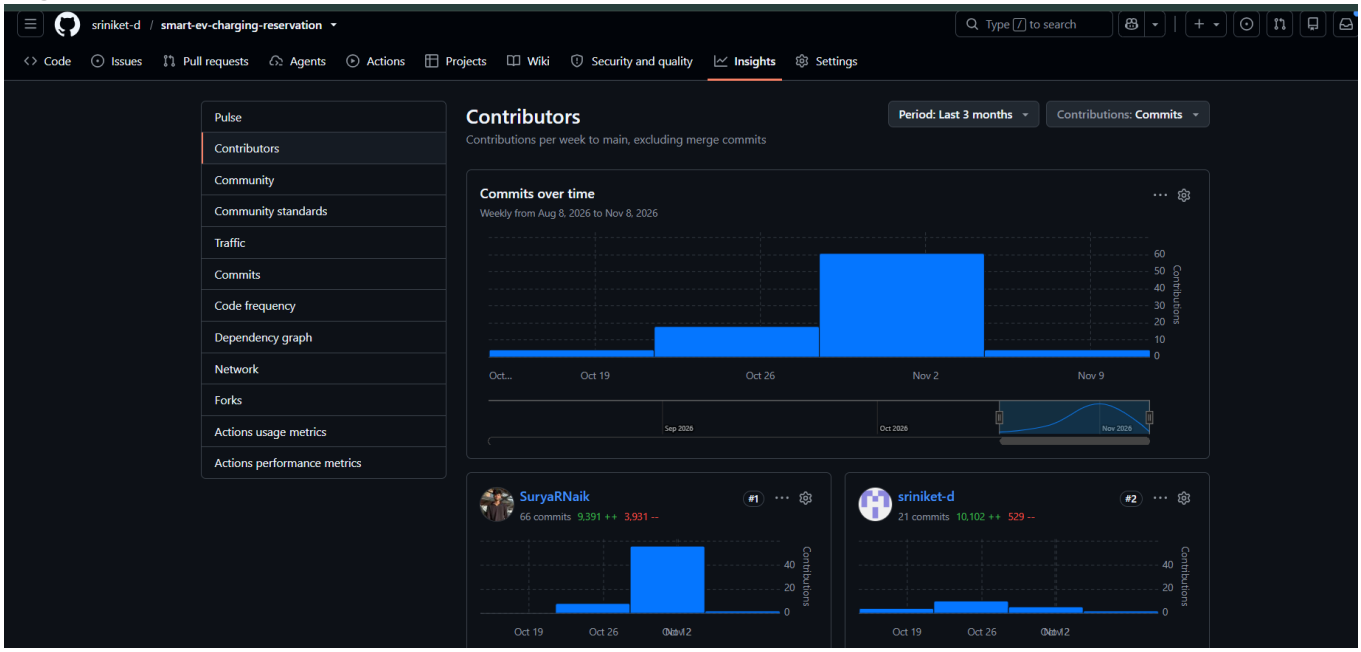


Figure 10 – Github contributors / commits



9. Evaluation Rubrics – Completion Status

Statuses are based on the repository contents (code, SQL scripts, docs/test-cases.md). The application was not executed while preparing this report; runtime evidence is the team's own recorded manual tests.

No.	Requirement	Status	Evidence
1	Project title / not simple CRUD	Completed	Reservation overlap prevention, transactions, audit log and rolebased admin go beyond CRUD. Uniqueness within the section is for the instructor to confirm.
2	Team formation (2 members)	Completed	TASK_DISTRIBUTION.md defines roles; git history has commits from both members (30 and 67).
3	Database requirement (SQL + NoSQL)	Completed	MySQL (mandatory) + MongoDB; justification in docs/projectoverview.md.
4	Database design (ER, schema, NoSQL model)	Completed	docs/er-diagram.md, docs/data-dictionary.md, db/schema.sql, docs/mongodb-data-model.md.
5	SQL implementation (DDL)	Completed	CREATE, ALTER (migration, temp table), DROP, PK/FK, constraints, indexes and a view in db/.
6	Database operations (DML, queries)	Completed	INSERT/UPDATE/DELETE/SELECT; JOIN, GROUP BY, HAVING, ORDER BY, aggregates, subquery.
7	Advanced features (4 SQL + 1 NoSQL)	Completed	SQL: transactions, view, triggers, indexing. NoSQL: aggregation. No stored procedures or full-text search.
8	Vector DB / semantic search (optional)	Not Implemented	No embeddings or vector store in the repository; not required for this domain.
9	Web interface (Next.js)	Completed	app/page.tsx, login and reservations pages: forms, tables, dashboards.
10	DB–application integration	Completed	Route Handlers use mysql2 pool and MongoDB driver; credentials from env vars; .env* git-ignored.
11	Authentication and authorization	Completed	bcrypt hashes, signed HttpOnly session, customer and admin roles (403 for non-admin on admin API), role re-checked against MySQL.
12	Data validation and integrity	Completed	CHECK, UNIQUE, NOT NULL, FK RESTRICT and overlap triggers at DB level plus server-side validation.
13	Project documentation	Completed	Docs cover all items; application screenshots and some test results are still to be inserted (Section 8).
14	Git / GitHub	Completed	Repository with PRs and commits from both members; README with setup steps.
15	Evaluation / demonstration readiness	Partially Completed	Test-case matrix exists, but several cases are marked Pending (e.g. PAY-02, ADMIN-02, ADMIN-05, ADMIN-08).
16	Originality	Completed	Project-specific schema, triggers and workflows; final judgement rests with the instructor.

17	Level 3 expectation	Completed	Two database paradigms, justified by data type, linked to a working web application.